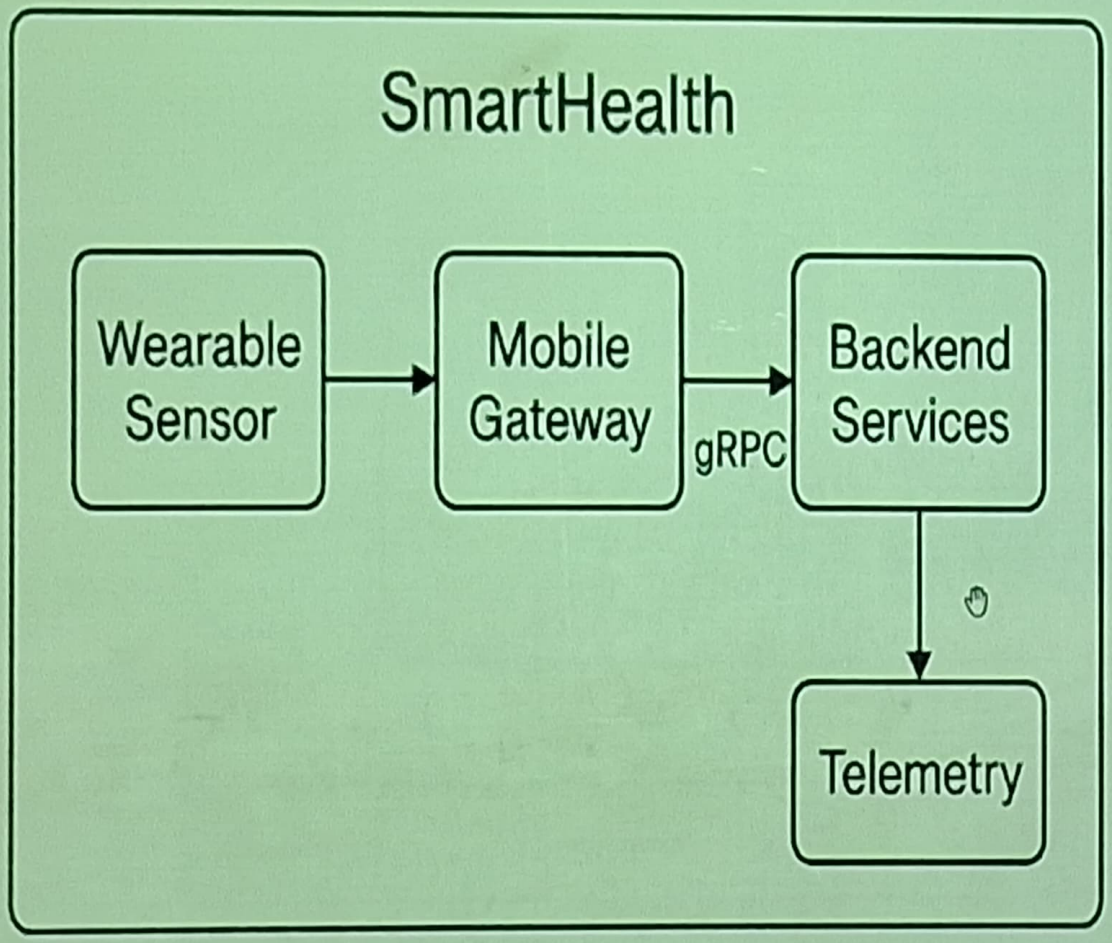
SmartHealth
Wearable Sensor
Mobile Gateway
gRPC
Backend Services
Telemetry

Requirements Gathering :Usability · Auditability · Accessibility

October 28, 2025

- Mobile + Web app capturing sensor data (steps, heart-rate, SpO2), performing analytics, showing recommendations.
- Core components:
  - Edge device: wearable sensors sampling at 1–50 Hz
  - Mobile gateway: Bluetooth LE ingestion, local buffering
  - Backend: gRPC microservices, store (timeseries DB), analytics pipeline (streaming)
  - Telemetry: Prometheus, Jaeger, ELK, Grafana dashboards
- Non-functional requirements: accuracy $\geq$95% (heart-rate), P95 latency $<$200 ms, availability 99.9%.

Prometheus

- Open-source **metrics monitoring** and alerting system.
- Collects time-series data using **PromQL** for flexible analysis.

Jaeger

- Tool for **distributed tracing** across microservices.
- Helps analyze **latency and request paths** end-to-end.

ELK Stack (Elasticsearch, Logstash, Kibana)

- Complete **log aggregation and analysis** suite.
- Logstash ingests → Elasticsearch indexes → Kibana visualizes.

Grafana Dashboards

- Unified **data visualization** and dashboard platform.
- Integrates with Prometheus, ELK, and others for **real-time monitoring**.

Prometheus

- Open-source **metrics monitoring** and alerting system.
- Collects time-series data using **PromQL** for flexible analysis.

Jaeger

- Tool for **distributed tracing** across microservices.
- Helps analyze **latency and request paths** end-to-end.

ELK Stack (Elasticsearch, Logstash, Kibana)

- Complete **log aggregation and analysis** suite.
- Logstash ingests → Elasticsearch indexes → Kibana visualizes.

Grafana Dashboards

- Unified **data visualization** and dashboard platform.
- Integrates with Prometheus, ELK, and others for **real-time monitoring**.

- Translate vague requirements into measurable criteria (S.M.A.R.T).
- Example: "responsive UI" $\rightarrow$ measurable: median response time, task completion time distribution.
- Each requirement needs: metric, unit, collection method, frequency, SLA/threshold.

- **Accuracy**: fraction of correct outputs vs ground truth.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision / Recall / F1** for classification.
- **Latency** (per-request): measure distribution: mean, median, P95, P99.
- **Throughput**: requests / second (RPS).
- **Resource Utilization**: CPU%, memory MB, I/O MB/s.

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Meaning: Of all items predicted positive, how many were truly positive?

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Meaning: Of all true positives, how many were correctly identified?

F1-Score

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Meaning: Harmonic mean of Precision and Recall — balances both metrics.

High Precision ⇒ few false positives; High Recall ⇒ few false negatives.

- Sensor data pipeline: sample timestamps, sample rate, calibration factor.
- Attach `trace_id` per user action to correlate UI → backend → DB.
- Telemetry types:
  - **Metrics** (Prometheus): counters, gauges, histograms.
  - **Traces** (Jaeger/OpenTelemetry): distributed spans.
  - **Logs** (structured JSON): event time, level, context.

Technical Usability Metrics

- **Task completion time** (seconds) + distribution.
- **Error rate**: number of failed tasks / attempts.
- **System Usability Scale (SUS)**: 10-question score $\in [0, 100]$.
- **Keystroke-Level Model (KLM)** / GOMS for expert time prediction.
- **Cognitive Load proxies**: pupil dilation (if hardware), task switching frequency.

- Define hypotheses, e.g.: "New layout reduces median task time by 20%".
- Choose sample size via power analysis:

  For two-sided t-test: $n = 2\left(\frac{z_{1-\alpha/2}+z_{1-\beta}}{\delta/\sigma}\right)^2$
- Randomized A/B assignment; collect task times, SUS.
- Analyze: t-test / Mann-Whitney U (non-parametric) + effect size (Cohen's d).

- Frontend collects:
  - event: `task_start`, `task_end` with timestamps
  - metadata: `user_id_hash`, `device`, `ui_version`
- Send to backend via batched / compressed protobuf message every 30s.

```
{
  "event": "task_end",
  "task_id": "log_activity",
  "elapsed_ms": 11500,
  "ui_version": "v2.1",
  "user_hash": "a3b9..."
}
```

- Input data provenance (timestamps, sensor calibration, sample rate).
- Model decisions and confidence scores (if ML used).
- Policy decisions (who changed thresholds, when).
- System performance metrics for SLA verification.

- Define workloads: realistic user traces vs synthetic bursts.
- Warm-up period to remove JIT/startup bias.
- Measure steady-state behavior: collect latency histograms (not just mean).
- Use percentiles (P50,P90,P95,P99,P999) and present CDFs.

Overview

`wrk` is a fast, modern HTTP benchmarking tool written in C.
It uses efficient I/O (epoll/kqueue) and multi-threading to generate high load on web servers.

Key Features

- Multi-threaded, scalable benchmarking
- Supports HTTP/HTTPS and custom Lua scripting
- Reports latency distribution, throughput, and transfer rates

Basic Usage

```
wrk -t4 -c100 -d30s http://localhost:8080
```

Parameters: 4 threads, 100 connections, 30 seconds duration

Use Cases

- Load testing of web servers and APIs

Profiling & Resource Analysis

- **CPU/Memory**: `perf`, `pprof`, eBPF tools (bcc, bpftrace).
- **I/O**: `iostat`, `iotop`, storage latency histograms.
- **GC**: heap dumps and pause analysis for managed runtimes.
- **Example: eBPF one-liner**:

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_read { @[comm]
```

- Perceivable: text alternatives, captions, adaptable layouts.
- Operable: keyboard navigation, focus order, timeouts alternatives.
- Understandable: clear language, consistent UI, ARIA roles.
- Robust: semantic HTML, proper role/aria attributes for assistive tech.

- Perceivable: text alternatives, captions, adaptable layouts.
- Operable: keyboard navigation, focus order, timeouts alternatives.
- Understandable: clear language, consistent UI, ARIA roles.
- Robust: semantic HTML, proper role/aria attributes for assistive tech.

```
<!-- Button with role and label -->
<button aria-label="Log my walk" id="log-walk">Log</button>

<!-- Accessible form -->
<label for="steps">Steps</label>
<input id="steps" name="steps" type="number" aria-required="true"/>
```